

Los Aromos: probar la reserva de canchas *de principio a fin*

En las actividades anteriores probaste piezas sueltas de código, sin navegador. Acá es al revés: la aplicación se levanta entera y tú escribes un programa que **la usa como la usaría un socio del complejo** — entra, filtra, reserva — y comprueba que hace lo que promete.

Cinco requisitos, **22 escenarios**. Todo con Cypress, sin trucos y sin conocimientos previos de pruebas: lo que necesitas está en la práctica en vivo de hoy y en este documento.

Este documento **no trae los escenarios resueltos**. Trae la aplicación que hay que probar, la lista de casos y los criterios con los que se revisa tu entrega.

EL ENCARGO

El encargo

El **Complejo Deportivo Los Aromos** arrendaba sus canchas por WhatsApp. Este año estrenó una aplicación: el socio entra con su correo, ve las canchas libres del día, filtra por tipo y reserva la que quiera.

Hay una regla del complejo, y es la que importa: **cada socio puede tomar como máximo dos canchas al día**. Sin eso, tres personas se llevan el recinto completo y el resto se queda afuera.

La aplicación está entregada y **no tiene ni una prueba**. Te contratan para cubrirla antes de que se abra al público el lunes.

CUIDADO

Hay un defecto escondido. Uno solo.

Está en la pantalla de canchas y tiene que ver con la **regla de las dos reservas**. No te voy a decir cuál es.

Si escribes los escenarios que se te piden, **uno de tus tests se va a poner rojo**. Cuando pase: **arreglas la aplicación, nunca el test**, y lo cuentas en el README.

Pista de método, no de contenido: cuando una regla dice «*máximo N*», en el código hay casi siempre una comparación con ese N. Los defectos de esta familia no viven en el número — viven en el **signo** de la comparación. Léela en voz alta antes de creerle.

PISTA

Qué se evalúa y qué no.

Se evalúa que tus escenarios **describan lo que hace una persona**, no cómo está construida la pantalla; que **ningún test dependa de otro**; y que **no haya ni una espera por tiempo**.

Se evalúa también que **todos los selectores sean** `data-cy`. Un test que pasa buscando por clase CSS igual se descuenta: pasa hoy y se cae en el próximo rediseño.

No se evalúa la cantidad de tests. Veintidós escenarios bien elegidos valen más que sesenta repetidos — en end-to-end, de más es un defecto, no una virtud.

RUTA DE TRABAJO

◆ De dónde partes

· proyecto, Cypress y la aplicación que hay que probar

1

Requisito 1 — Abrir y mirar

· `visit`, `get`, `should` · 4 escenarios

2

Requisito 2 — El acceso

· `type`, `click`, `url` · 5 escenarios

3

Requisito 3 — La lista y el filtro

· `select`, `each` · 5 escenarios

4

Requisito 4 — Cuando el servidor falla



· `cy.intercept` · 5 escenarios

5

Requisito 5 — Comandos propios y la regla del negocio

· 3 escenarios · acá está el defecto



Cómo se revisa · Errores que te van a pasar · Si terminas antes

◆ De dónde partes

Antes de escribir un solo `it`. Crea el proyecto, pega los archivos que te entregamos y comprueba que el escenario de ejemplo pasa en verde.

> ARCHIVOS QUE TE ENTREGAMOS EN ESTA SECCIÓN

11 archivos

- 1 vite.config.js
- 2 src/servicios/api.js
- 3 src/sesion.js
- 4 src/router.js
- 5 src/vistas/VistaAcceso.vue
- 6 src/vistas/VistaCanchas.vue
- 7 src/main.js
- 8 src/App.vue
- 9 cypress.config.js
- 10 package.json
- 11 cypress/e2e/00-ejemplo.cy.js

Éstos vienen listos: cópialos tal cual. Lo que tú escribes son los archivos de cypress/e2e/, que no están acá.

1 Paso 1 — El proyecto y las dependencias

> Terminal

```
# Crea la carpeta con una app Vue basica ya adentro
npm create vite@latest reservas-e2e -- --template vue
cd reservas-e2e
npm install

# vue-router: Le da direccion propia a cada pantalla (/acceso, /canchas)
npm install vue-router

# Cypress. El -D es "dependencia de desarrollo": herramienta tuya,
```

no viaja al producto que ve el socio del club.

```
npm install -D cypress
```

PISTA

La primera instalación de Cypress descarga unos 200 MB.

Trae su propio navegador. Se baja una sola vez y queda guardada en tu sistema, no dentro de `node_modules`. Si estás en clase con el wifi compartido, **parte por acá** y sigue leyendo mientras baja.

2 Paso 2 — La aplicación (te la damos entera)

Siete archivos. **Cópioslos tal cual** — hoy no se evalúa que sepas construir la aplicación, se evalúa que sepas probarla. De cada uno te decimos en una línea qué hace.

La configuración de Vite, con el servidor de datos de la clase adentro. Ese servidor guarda las canchas en memoria y responde en `/api`. Fíjate en `/api/reset`: es la dirección que te va a permitir que cada escenario parta desde el mismo punto.

✓ vite.config.js **TE LO DAMOS**

```
import { fileURLToPath, URL } from 'node:url'
import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'

// — SERVIDOR DE DATOS DE LA CLASE —
// Cuatro canchas libres y un contador de reservas del día.
// Todo en memoria: al apagar el servidor se olvida.
const SEMILLA = () => ([
  { id: 'C-01', nombre: 'Cancha 1', tipo: 'futbolito', hora: '19:00', valor: 24000 },
  { id: 'C-02', nombre: 'Cancha 2', tipo: 'futbolito', hora: '20:00', valor: 24000 },
  { id: 'C-03', nombre: 'Cancha 3', tipo: 'padel', hora: '19:30', valor: 16000 },
  { id: 'C-04', nombre: 'Cancha 4', tipo: 'padel', hora: '21:00', valor: 18000 }
])

function apiDeClase() {
  let canchas = SEMILLA()
  let reservasDelDia = 0

  const leerCuerpo = (req) => new Promise((listo) => {
```

```

let crudo = ''
req.on('data', (trozo) => { crudo += trozo })
req.on('end', () => listo(crudo ? JSON.parse(crudo) : {}))
})

return {
  name: 'api-de-clase',
  configureServer(server) {
    // AL montar en '/api' el prefijo se recorta:
    // /api/canchas llega aca como /canchas.
    server.middlewares.use('/api', async (req, res, next) => {
      const ruta = req.url.split('?')[0]
      res.setHeader('Content-Type', 'application/json')

      if (req.method === 'POST' && ruta === '/acceso') {
        const { correo, clave } = await leerCuerpo(req)
        const ok = correo === 'vecino@losaromos.cl' && clave === 'cancha2026'
        res.statusCode = ok ? 200 : 401
        return res.end(JSON.stringify(
          ok ? { token: 'token-de-clase', nombre: 'Rosa Miranda' }
            : { mensaje: 'Correo o clave incorrectos' }
        ))
      }

      if (req.method === 'GET' && ruta === '/canchas') {
        return res.end(JSON.stringify({ canchas, reservasDelDia }))
      }

      if (req.method === 'POST' && /^\/canchas\/[^\s]+\/reservar$/.test(ruta)) {
        const id = ruta.split('/')[2]
        canchas = canchas.filter((c) => c.id !== id)
        reservasDelDia = reservasDelDia + 1
        return res.end(JSON.stringify({ ok: true, reservasDelDia }))
      }

      // Deja todo como recién instalado.
      if (req.method === 'POST' && ruta === '/reset') {
        canchas = SEMILLA()
        reservasDelDia = 0
        return res.end(JSON.stringify({ ok: true }))
      }

      next()
    })
  }
}

```

```

    }
  }
}

export default defineConfig({
  plugins: [vue(), apiDeClase()],
  resolve: {
    alias: { '@': fileURLToPath(new URL('./src', import.meta.url)) }
  },
  // Puerto fijo: si Vite se cambia solo al 5174, la baseUrl de Cypress
  // apunta a ninguna parte y todos tus tests fallan igual.
  server: { port: 5173, strictPort: true }
})

```

La única salida a la red. Todo lo que la aplicación le pide al servidor pasa por acá. Que sea una sola puerta es lo que después te va a permitir interceptarla desde un test.

✓ src/servicios/api.js **TE LO DAMOS**

```

const BASE = '/api'

export async function entrar(correo, clave) {
  const respuesta = await fetch(`${BASE}/acceso`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ correo, clave })
  })
  const datos = await respuesta.json()
  if (!respuesta.ok) throw new Error(datos.mensaje || 'No pudimos validar tus dato')
  return datos
}

export async function obtenerCanchas() {
  const respuesta = await fetch(`${BASE}/canchas`)
  if (!respuesta.ok) throw new Error('No pudimos cargar las canchas')
  return respuesta.json()
}

export async function reservarCancha(id) {
  const respuesta = await fetch(`${BASE}/canchas/${id}/reservar`, { method: 'POST' })
  if (!respuesta.ok) throw new Error('No pudimos tomar la reserva')
  return respuesta.json()
}

```

Recuerda quién entró. `localStorage` es una libretita que el navegador guarda por cada sitio y que sobrevive a un F5.

✓ src/sesion.js **TE LO DAMOS**

```
import { ref } from 'vue'

export const socio = ref(JSON.parse(localStorage.getItem('sesion-aromos')) || 'null')

export function guardarSesion(datos) {
  socio.value = datos
  localStorage.setItem('sesion-aromos', JSON.stringify(datos))
}

export function cerrarSesion() {
  socio.value = null
  localStorage.removeItem('sesion-aromos')
}
```

Decide qué pantalla se ve según la dirección. Al final hay un guardia: una función que corre antes de cada cambio de pantalla y devuelve al acceso a quien no tenga sesión.

✓ src/router.js **TE LO DAMOS**

```
import { createRouter, createWebHistory } from 'vue-router'
import VistaAcceso from '@/vistas/VistaAcceso.vue'
import VistaCanchas from '@/vistas/VistaCanchas.vue'
import { socio } from '@/sesion.js'

export const router = createRouter({
  history: createWebHistory(),
  routes: [
    { path: '/', redirect: '/canchas' },
    { path: '/acceso', name: 'acceso', component: VistaAcceso },
    { path: '/canchas', name: 'canchas', component: VistaCanchas, meta: { privada: true } }
  ]
})

router.beforeEach((destino) => {
```



```
    if (destino.meta.privada && !socio.value) return { name: 'acceso' }  
  })
```

La pantalla de acceso. Mírala entre `<template>` y `</template>`: ahí están los `data-cy` que vas a usar.

✓ src/vistas/VistaAcceso.vue **TE LO DAMOS**

```
<script setup>  
import { ref } from 'vue'  
import { useRouter } from 'vue-router'  
import { entrar } from '@servicios/api.js'  
import { guardarSesion } from '@sesion.js'  
  
const correo = ref('')  
const clave = ref('')  
const error = ref('')  
const enviando = ref(false)  
const router = useRouter()  
  
async function enviar() {  
  error.value = ''  
  enviando.value = true  
  try {  
    guardarSesion(await entrar(correo.value, clave.value))  
    router.push('/canchas')  
  } catch (e) {  
    error.value = e.message  
  } finally {  
    enviando.value = false  
  }  
}  
</script>  
  
<template>  
  <main class="acceso">  
    <h1>Complejo Los Aromos</h1>  
    <p>Reserva tu cancha del día</p>  
  
    <form data-cy="form-acceso" @submit.prevent="enviar">  
      <label for="correo">Correo</label>  
      <input id="correo" data-cy="correo" v-model="correo" type="email" required>
```

```

<label for="clave">Clave</label>
<input id="clave" data-cy="clave" v-model="clave" type="password" required>

<button data-cy="entrar" type="submit" :disabled="enviando">
  {{ enviando ? 'Entrando...' : 'Entrar' }}
</button>
</form>

<p v-if="error" data-cy="error-acceso" role="alert">{{ error }}</p>
</main>
</template>

```

La pantalla de canchas. Es la importante. Tiene el filtro, los cuatro estados y la regla de las dos reservas.

✓ src/vistas/VistaCanchas.vue **TE LO DAMOS**

```

<script setup>
import { computed, onMounted, ref } from 'vue'
import { useRouter } from 'vue-router'
import { obtenerCanchas, reservarCancha } from '@/servicios/api.js'
import { cerrarSesion, socio } from '@/sesion.js'

const LIMITE_DIARIO = 2

const canchas = ref([])
const reservasDelDia = ref(0)
const cargando = ref(false)
const error = ref('')
const tipo = ref('todas')
const router = useRouter()

const visibles = computed(() =>
  tipo.value === 'todas'
    ? canchas.value
    : canchas.value.filter((c) => c.tipo === tipo.value)
)

// Regla del complejo: cada socio toma como maximo 2 canchas al dia.
const limiteAlcanzado = computed(() => reservasDelDia.value > LIMITE_DIARIO)

async function cargar() {
  cargando.value = true

```

```

error.value = ''
try {
  const datos = await obtenerCanchas()
  canchas.value = datos.canchas
  reservasDelDia.value = datos.reservasDelDia
} catch {
  error.value = 'No pudimos cargar las canchas. Intenta nuevamente.'
} finally {
  cargando.value = false
}
}

async function reservar(id) {
  const datos = await reservarCancha(id)
  reservasDelDia.value = datos.reservasDelDia
  canchas.value = canchas.value.filter((c) => c.id !== id)
}

function salir() {
  cerrarSesion()
  router.push('/acceso')
}

onMounted(cargar)
</script>

<template>
  <main class="canchas">
    <header>
      <h1>Canchas disponibles hoy</h1>
      <p data-cy="socio">{{ socio?.nombre }}</p>
      <p data-cy="contador">Reservas de hoy: {{ reservasDelDia }} de {{ LIMITE_DIA }}
      <button data-cy="salir" type="button" @click="salir">Salir</button>
    </header>

    <label for="tipo">Tipo de cancha</label>
    <select id="tipo" data-cy="filtro-tipo" v-model="tipo">
      <option value="todas">Todas</option>
      <option value="futbolito">Futbolito</option>
      <option value="padel">Padel</option>
      <option value="tenis">Tenis</option>
    </select>

    <p v-if="limiteAlcanzado" data-cy="aviso-limite">
      Ya tomaste tus {{ LIMITE_DIARIO }} canchas de hoy.

```

```

</p>

<p v-if="cargando" data-cy="cargando">Cargando las canchas...</p>

<div v-else-if="error" data-cy="error-canchas" role="alert">
  <p>{{ error }}</p>
  <button data-cy="reintentar" type="button" @click="cargar">Reintentar</button>
</div>

<p v-else-if="visibles.length === 0" data-cy="vacio">
  No hay canchas de ese tipo disponibles.
</p>

<ul v-else data-cy="lista">
  <li v-for="c in visibles" :key="c.id" data-cy="cancha" :data-id="c.id">
    <span data-cy="nombre">{{ c.nombre }}</span>
    <span data-cy="tipo">{{ c.tipo }}</span>
    <span data-cy="hora">{{ c.hora }}</span>
    <span data-cy="valor">${{{ c.valor }}}</span>
    <button
      data-cy="reservar"
      type="button"
      :disabled="limiteAlcanzado"
      @click="reservar(c.id)"
    >Reservar</button>
  </li>
</ul>
</main>
</template>

```

El almacén. Arrancan la aplicación y muestran la pantalla que toque. Cópialos y olvídate.

✓ src/main.js **TE LO DAMOS**

```

import { createApp } from 'vue'
import App from './App.vue'
import { router } from './router.js'
import './style.css'

createApp(App).use(router).mount('#app')

```

✓ src/App.vue TE LO DAMOS

```
<template>
  <RouterView />
</template>
```

3

Paso 3 — Cypress

Necesitas **dos terminales abiertas al mismo tiempo**: en la primera dejas la aplicación funcionando y no la cierras en toda la actividad; en la segunda abres Cypress.

> Terminal 1 – la aplicación

```
npm run dev

# → Local: http://localhost:5173/
```

> Terminal 2 – Cypress

```
npx cypress open
```

Elige **E2E Testing**, deja que genere los archivos que te propone y elige el navegador. Te va a quedar creada la carpeta `cypress/`. Después reemplaza la configuración por ésta:

✓ cypress.config.js TE LO DAMOS

```
import { defineConfig } from 'cypress'

export default defineConfig({
  e2e: {
    // Con esto, cy.visit('/acceso') ya sabe a donde ir
    baseUrl: 'http://localhost:5173',

    specPattern: 'cypress/e2e/**/*.cy.js',
    supportFile: 'cypress/support/e2e.js',
```

```
    viewportWidth: 1280,  
    viewportHeight: 800,  
    defaultCommandTimeout: 4000,  
    video: false  
  }  
})
```

✓ package.json **TE LO DAMOS**

```
{  
  "scripts": {  
    "dev": "vite",  
    "build": "vite build",  
  
    "cy:open": "cypress open",  
    "cy:run": "cypress run"  
  }  
}
```

4 Paso 4 — El escenario de ejemplo, resuelto

Éste te lo damos escrito para que veas la forma. Los **22 que vienen después los escribes tú**.

✓ cypress/e2e/00-ejemplo.cy.js **TE LO DAMOS**

```
describe('El complejo esta en pie', () => {  
  it('la pantalla de acceso se abre en el navegador', () => {  
    cy.visit('/acceso')  
    cy.contains('h1', 'Complejo Los Aromos').should('be.visible')  
  })  
  
  it('el servidor de canchas responde', () => {  
    // cy.request golpea el servidor directo, sin pasar por la pantalla.  
    // Sirve para comprobar el backend y para preparar datos.  
    cy.request('/api/canchas')  
      .its('status')  
      .should('eq', 200)  
  })  
})
```

```
})  
})
```

> Terminal 2

```
npm run cy:run
```

```
El complejo esta en pie
```

```
✓ la pantalla de acceso se abre en el navegador (588ms)
```

```
✓ el servidor de canchas responde (41ms)
```

```
2 passing (1s)
```

5 Paso 5 — Los datos con los que vas a trabajar

Todos tus escenarios parten de este estado. Apréndetelo: la mitad de tus aserciones son números que salen de esta tabla.

DATA-ID	NOMBRE	TIPO	HORA	VALOR
C-01	Cancha 1	futbolito	19:00	\$24000
C-02	Cancha 2	futbolito	20:00	\$24000
C-03	Cancha 3	padel	19:30	\$16000
C-04	Cancha 4	padel	21:00	\$18000

CLAVE

Fíjate: hay futbolito y hay pádel, pero no hay ni una cancha de tenis.

El filtro sí tiene la opción «Tenis». Eso no es un descuido del enunciado: es el **estado vacío**, y es uno de los escenarios que tienes que probar.

↔ Lo que responde GET /api/canchas

```
{
  "reservasDelDia": 0,
  "canchas": [
    { "id": "C-01", "nombre": "Cancha 1", "tipo": "futbolito", "hora": "19:00", "v
```

PISTA

Anota esa forma: es un objeto, no un arreglo.

Cuando en el Requisito 4 tengas que dictarle la respuesta al servidor, tu respuesta falsa tiene que tener **exactamente esta forma** — con las dos llaves, `canchas` y `reservasDelDia`. Si le entregas un arreglo pelado, la pantalla revienta y vas a creer que Cypress está fallando.

6 Paso 6 — Las credenciales y las direcciones

QUÉ	VALOR
Correo del socio	<code>vecino@losaromos.cl</code>
Clave	<code>cancha2026</code>
Nombre que muestra la pantalla	Rosa Miranda
Pantalla de acceso	<code>/acceso</code>
Pantalla de canchas (protegida)	<code>/canchas</code>
Dejar los datos como al principio	<code>cy.request('POST', '/api/reset')</code>

CRITERIOS

No sigas hasta que se cumpla esto

- ☐ `npm run dev` muestra la pantalla de acceso en `localhost:5173`.
- ☐ Entrás con las credenciales de arriba y ves las 4 canchas.

- ☐ `npm run cy:run` pasa los 2 escenarios de ejemplo.
- ☐ Entiendes por qué necesitas dos terminales.
- ☐ Sabes para qué existe `/api/reset`.

1 Requisito 1 — Abrir y mirar

Entregable: `cypress/e2e/01-primeros-pasos.cy.js` · **4 escenarios** · Sin escribir en ningún campo todavía.

1 Qué se necesita

Antes de manejar la aplicación hay que saber **mirarla**. Estos cuatro escenarios no tocan nada: abren una dirección y comprueban qué se ve. Es el piso de todo lo demás.

#	SITUACIÓN	QUÉ TIENES QUE COMPROBAR
1	Visito <code>/acceso</code>	El título <code><h1></code> «Complejo Los Aromos» se ve en pantalla
2	Visito <code>/acceso</code>	El formulario se ve, y los campos de correo y clave parten vacíos
3	Visito <code>/acceso</code>	El botón dice « Entrar » y está habilitado
4	Visito <code>/canchas</code> sin haber entrado	Termino en <code>/acceso</code> , con el formulario a la vista

2 Las herramientas que necesitas

ORDEN	QUÉ HACE
<code>cy.visit('/acceso')</code>	Abre esa dirección, como escribirla en la barra y apretar Enter
<code>cy.get('[data-cy="..."]')</code>	Busca un elemento por su manilla
<code>cy.contains('h1', 'texto')</code>	Busca un <code><h1></code> cuyo texto incluya eso
<code>cy.url()</code>	La dirección completa del navegador en este momento
<code>.should('be.visible')</code>	Existe y se ve

ORDEN	QUÉ HACE
<code>.should('have.value', '')</code>	El contenido de un campo de formulario
<code>.should('be.enabled')</code>	El control se puede usar

CLAVE

El escenario 4 es el que le da sentido a toda la actividad.

Esa regla —«sin sesión no entras»— no vive en la pantalla de canchas ni en el archivo de sesión: vive en **la unión** entre el guardia del router y el almacenamiento del navegador. Ninguna prueba de una pieza suelta la habría visto. Ésa es, literalmente, la razón de existir de las pruebas end-to-end.

PISTA

Sobre el escenario 4: no basta con mirar la dirección.

Un test que solo comprueba la URL puede pasar aunque la pantalla haya quedado en blanco. Comprueba las **dos** cosas: que la dirección es la del acceso y que el formulario se ve.

3 Criterios de aceptación

CRITERIOS

Se revisa esto, uno por uno

- ☐ Los 4 escenarios están dentro de **un solo** `describe` con nombre en castellano.
- ☐ Cada `it` se llama por lo que **hace el usuario**, no por el comando que usaste. «muestra el formulario de acceso», no «test cy.get».
- ☐ Todos los selectores son `data-cy`. Cero clases CSS, cero `nth-child`.
- ☐ El escenario 2 usa `have.value` con la cadena vacía, no `be.empty`.
- ☐ El escenario 4 comprueba la dirección **y** que el formulario se ve.
- ☐ No hay ni un `cy.wait` con un número.
- ☐ La visita a la dirección está repetida en cada `it`, o sacada a un `beforeEach`. Las dos valen; lo que no vale es que un `it` dependa de que otro haya visitado antes.

CUIDADO

El error silencioso que te va a costar puntos: guardar el resultado en una variable. `const titulo = cy.get('h1')` no te devuelve el título — te devuelve la cadena de Cypress. La comprobación va **pegada con un punto:**

`cy.get('h1').should(...).`

2 Requisito 2 — El acceso

Entregable: `cypress/e2e/02-acceso.cy.js` · 5 escenarios · Acá empiezas a manejar la aplicación.

1 Qué se necesita

El trámite completo de entrar: con las credenciales buenas, con las malas, con el teclado y saliendo. Es el flujo que más se usa y el que menos se prueba.

#	SITUACIÓN	QUÉ TIENES QUE COMPROBAR
1	Escribo <code>vecino@losaromos.cl / cancha2026</code> y hago clic en «Entrar»	Llego a <code>/canchas</code> , se ve Rosa Miranda y hay 4 canchas
2	Escribo un correo que no existe y una clave cualquiera	Aparece el mensaje « Correo o clave incorrectos », sigo en <code>/acceso</code> y el formulario se ve
3	Ese mismo intento fallido	El mensaje no contiene el número 401 ni la palabra «error» en inglés
4	Escribo las credenciales buenas y aprieto Enter , sin tocar el botón	Entro igual: Llego a <code>/canchas</code>
5	Ya adentro, hago clic en «Salir»	Vuelvo a <code>/acceso</code> , y si intento visitar <code>/canchas</code> me devuelve otra vez

2 Las herramientas nuevas

ORDEN	QUÉ HACE
<code>.type('texto')</code>	Escribe tecla por tecla, como una persona
<code>.type('clave{enter}')</code>	Escribe y después aprieta Enter — así se resuelve el escenario 4

ORDEN	QUÉ HACE
<code>.click()</code>	Hace clic
<code>.should('contain', 'texto')</code>	El texto está dentro (parcial)
<code>.should('not.contain', '401')</code>	El texto no está
<code>.and(...)</code>	Una segunda comprobación sobre lo mismo, para no repetir el <code>cy.get</code>
<code>.should('not.exist')</code>	El elemento no está en la página

CLAVE

El escenario 3 parece un capricho y es el más profesional de los cinco.

Un mensaje que le tira al socio del club un «Error 401» no le sirve de nada: no sabe qué hacer con eso. Comprobar que *no* aparece el número técnico es comprobar que alguien tradujo el fallo a lenguaje humano. Eso es calidad de producto, y se prueba igual que todo lo demás.

PISTA

Pista sobre el escenario 5.

«Salir» tiene que hacer dos cosas: llevarte a `/acceso` y borrar la sesión. Si solo comprobaras la primera, un botón que navega pero se olvida de borrar pasaría el test — y cualquiera que escriba `/canchas` en la barra entraría con la sesión del anterior. Por eso el escenario pide *volver a intentar* entrar a `/canchas`.

3

Criterios de aceptación

CRITERIOS

Se revisa esto, uno por uno

- ☐ Hay un `beforeEach` que deja el sistema en estado conocido: `cy.request('POST', '/api/reset')`.

- ☐ El escenario 4 usa `{enter}` dentro del `.type()`. **No** hace clic en el botón.
- ☐ El escenario 2 comprueba **las tres** cosas: mensaje, dirección y formulario visible.
- ☐ El escenario 5 comprueba que `/canchas` sigue protegida *después* de salir.
- ☐ Usas `.and()` al menos una vez, en vez de repetir el mismo `cy.get`.
- ☐ Ningún escenario depende de otro: puedes correr cualquiera solo y pasa igual.
- ☐ Sigue sin haber ni un `cy.wait` con un número.

CUIDADO

Te vas a topar con esto: después del clic en «Entrar» pasan varias cosas —cambia la pantalla, sale la petición, llega la respuesta, se dibuja la lista— y todo eso demora. La tentación es meter una espera. **No la metas.** Cypress ya vuelve a mirar la página hasta que tu comprobación se cumple. Si te sale rojo, el problema es tu selector, no el tiempo.

3 Requisito 3 — La lista y el filtro

Entregable: `cypress/e2e/03-lista.cy.js` · 5 escenarios · Acá aparece el estado vacío.

1 Qué se necesita

La pantalla de canchas con datos reales: contarlas, revisarlas una por una y filtrarlas por tipo. Acuérdate de la tabla del [Paso 5](#): 4 canchas, 2 de futbolito, 2 de pádel, **ninguna de tenis**.

#	SITUACIÓN	QUÉ TIENES QUE COMPROBAR
1	Entro y llego a la lista	Hay exactamente 4 canchas
2	Reviso la lista completa	Cada cancha muestra nombre, tipo, hora y valor no vacíos , y su botón «Reservar» está habilitado
3	Elijo Padel en el filtro	Quedan 2 , y son <code>C-03</code> y <code>C-04</code>
4	Elijo Tenis en el filtro	No queda ninguna cancha y aparece el estado vacío
5	Vuelvo a Todas	Vuelven las 4 : el filtro esconde, no borra

2 Las herramientas nuevas

ORDEN	QUÉ HACE
<code>.select('padel')</code>	Elige una opción de un <code><select></code> . Va el valor de la opción, no lo que se lee en pantalla
<code>.should('have.length', 4)</code>	Cuántos elementos coinciden con el selector
<code>.each((fila) => { ... })</code>	Recorre todos los encontrados, uno por uno

ORDEN	QUÉ HACE
<code>cy.wrap(fila)</code>	Le devuelve los poderes de Cypress al elemento que recibiste dentro del <code>.each()</code>
<code>.should('not.be.empty')</code>	El elemento tiene algo escrito adentro
<code>[data-cy="cancha"][data-id="C-03"]</code>	Apunta a una cancha concreta sin contar posiciones

CUIDADO

Ojo con los valores del filtro.

En pantalla se lee «Padel», pero el valor de la opción es `padel` — en minúscula. `.select()` quiere el valor. Si le pasas lo que se ve, Cypress te va a decir que esa opción no existe.

CLAVE

Por qué el escenario 4 vale doble.

El estado vacío es de los que nadie prueba a mano —hay que buscar una combinación que no devuelva nada— y de los que más se rompen. Una pantalla que en vez de decir «no hay canchas de ese tipo» se queda en blanco parece *caída*, y el socio cierra la aplicación creyendo que no anda.

Comprueba las dos caras: que **no** hay ninguna tarjeta y que el mensaje **sí** se ve. Una sola de las dos no alcanza.

PISTA

Pista sobre el escenario 2.

Dentro del `.each()`, la variable que recibes **no es** una cadena de Cypress: es el elemento pelado. Si le encadenas un `.find()` directo, no espera, no reintenta y no aparece en el panel. Envuélvelo primero.

Se revisa esto, uno por uno

- ☐ El escenario 2 usa `.each()` con `cy.wrap()` adentro. **No** repite cuatro bloques copiados.
- ☐ El escenario 3 identifica las canchas por `data-id`, no por `.first()`, `.eq()` ni posición.
- ☐ El escenario 4 comprueba **las dos** caras: ninguna tarjeta y el mensaje visible.
- ☐ Para «no hay ninguna tarjeta» usas `not.exist`, no `not.be.visible`.
- ☐ El `.select()` recibe el valor de la opción (`padel`, `tenis`), no el texto que se lee.
- ☐ El `beforeEach` reinicia los datos y entra. Los 5 escenarios parten del mismo punto.
- ☐ El escenario 5 demuestra que filtrar no pierde datos.

CUIDADO

La diferencia que te va a morder: cuando un elemento desaparece con `v-if`, **se va de la página** → `not.exist`. Cuando solo queda escondido, **sigue ahí** → `not.be.visible`. Elegir el equivocado te da un test rojo sin que nada esté roto, y vas a perder media hora buscando el error donde no está.

4 Requisito 4 — Cuando el servidor falla

Entregable: `cypress/e2e/04-servidor.cy.js` · 5 escenarios · Aquí aparece `cy.intercept`.

1 Qué se necesita

Hasta ahora el servidor siempre te contestó bien. En la vida real se cae, demora, o devuelve una lista vacía. **Ésos son los estados que nadie prueba** —porque cuesta provocarlos a mano— y son los que se rompen. Con `cy.intercept` cada uno sale en una línea.

#	SITUACIÓN	QUÉ TIENES QUE COMPROBAR
1	Todo normal, pero escuchando la petición de canchas	La respuesta llega con estado 200 y después se ven las 4 canchas
2	El servidor se cae: contesta 500	Se ve el mensaje de error, hay botón «Reintentar», no hay ninguna cancha y el mensaje no contiene «500»
3	Después de ese error, hago clic en «Reintentar» y ahora el servidor sí responde	El error desaparece y se ven las 3 canchas del archivo de datos
4	El servidor responde bien, pero sin ninguna cancha libre	Aparece el estado vacío y ninguna tarjeta
5	El servidor se demora 1,5 segundos en contestar	Antes se ve «Cargando las canchas...»; después de que contesta, ya no

2 Cómo se usa `cy.intercept`

la pantalla pide → Cypress escucha →

deja pasar · o contesta él mismo · o finge un error → la pantalla recibe

LO QUE QUIERES	CÓMO SE ESCRIBE	¿LLEGA AL SERVIDOR?
Escuchar y poder esperar	<code>cy.intercept('GET', '/api/canchas').as('cargar')</code>	Sí, pasa de largo
Contestar con datos fijos	<code>cy.intercept('GET', '/api/canchas', { body: { ... } })</code>	No
Contestar desde un archivo	<code>cy.intercept(..., { fixture: 'canchas-tarde.json' })</code>	No
Fingir que se cayó	<code>cy.intercept(..., { statusCode: 500 })</code>	No
Fingir que anda lento	<code>cy.intercept(..., { delay: 1500, body: { ... } })</code>	No
Esperar a que termine	<code>cy.wait('@cargar')</code>	—

CUIDADO

El orden no es negociable: interceptar ANTES de provocar la petición.

`cy.intercept` instala un oyente. Una petición que salió antes de que el oyente existiera es una petición que Cypress nunca pilló, y tu `cy.wait('@alias')` se va a quedar esperando hasta rendirse, con el mensaje «*no request ever occurred*».

En estos cinco escenarios eso significa: **primero el `cy.intercept`, después el `cy.visit` y el acceso**. Nunca al revés.

3 El archivo de datos que te damos

Para el escenario 3 necesitas una respuesta falsa distinta a la semilla. Cuando los datos inventados pasan de dos líneas, no se escriben dentro del test: se guardan en `cypress/fixtures/`. Crea este archivo tal cual.

```
{
  "reservasDelDia": 0,
  "canchas": [
    { "id": "C-07", "nombre": "Cancha 7", "tipo": "padel", "hora": "16:00", "v
    { "id": "C-08", "nombre": "Cancha 8", "tipo": "padel", "hora": "17:00", "v
    { "id": "C-09", "nombre": "Cancha 9", "tipo": "futbolito", "hora": "16:30", "v
  ]
}
```

PISTA**Para el escenario 4 no necesitas archivo.**

Una lista vacía cabe en una línea. Acuérdate de la forma que anotaste en la preparación: es un **objeto con dos llaves**. Si le entregas un arreglo pelado, la pantalla revienta y vas a creer que Cypress está fallando.

CLAVE**Escenario 5: por qué el retardo es la única forma de probar el «Cargando...».**

En tu computador el servidor de clase contesta en dos milisegundos: ese mensaje aparece y desaparece antes de que alcances a verlo, y un test tampoco alcanza. Con `delay: 1500` le das a la pantalla tiempo suficiente para mostrarlo, y recién ahí se puede comprobar. No es hacer trampa — es la única manera.

4

Criterios de aceptación**CRITERIOS****Se revisa esto, uno por uno**

- ☐ Todos los `cy.intercept` están **antes** del `cy.visit`.
- ☐ El escenario 1 usa un `cy.intercept` **sin respuesta** — solo escucha — y espera el alias.
- ☐ El escenario 2 comprueba **las cuatro** cosas: mensaje visible, botón «Reintentar», ninguna cancha, y que no aparezca el número 500.

- ☐ El escenario 3 declara una **segunda** interceptación sobre la misma dirección, después del error. Ése es el mecanismo, no un truco.
- ☐ El escenario 3 usa `fixture`, no un objeto escrito a mano en el test.
- ☐ El escenario 4 le entrega un objeto con `canchas` y `reservasDe1Dia`, no un arreglo.
- ☐ El escenario 5 comprueba el «Cargando...» **antes** del `cy.wait` y su ausencia **después**.
- ☐ Cero `cy.wait` con número. Todas las esperas son por alias.

CUIDADO

Si un escenario se queda pegado hasta que Cypress lo mata, casi siempre es una de dos: interceptaste después de la petición, o le pusiste el alias a una dirección que no coincide con la que sale de verdad. Abre el panel de `cy:open` y mira la lista de peticiones: ahí se ve cuál salió y cuál escuchaste.

5 Requisito 5 — Comandos propios y la regla del negocio

Entregables: `cypress/support/commands.js` y `cypress/e2e/05-reservas.cy.js` · 3 escenarios · [Acá está el defecto.](#)

1 Primero: deja de repetirte

A estas alturas todos tus `beforeEach` se parecen: reiniciar datos, visitar, escribir correo, escribir clave, entrar. Eso es **ruido**: quien lee el test tiene que saltarse cinco líneas que no dicen nada sobre lo que se está probando.

Cypress deja que te inventes tus propias órdenes. Le enseñas una vez qué significa «entrar» y después escribes `cy.entrar()` en cualquier test, como si viniera de fábrica.

■ Forma de un comando propio

```
Cypress.Commands.add('nombreDelComando', (parametro) => {  
  // Ojo con el `return`: sin el, la cadena se corta y no puedes  
  // encadenar .click() ni .should() despues.  
  return cy.get(`[data-cy="${parametro}"]`)  
})
```

Tu trabajo en esta parte es escribir **tres comandos** en `cypress/support/commands.js`, y comprobar que `cypress/support/e2e.js` lo importa.

COMANDO	QUÉ TIENE QUE HACER
<code>cy.reiniciarDatos()</code>	Dejar el servidor como recién instalado, con las 4 canchas y el contador en 0
<code>cy.entrar()</code>	Hacer el acceso completo con las credenciales del socio y no volver hasta estar en <code>/canchas</code>
<code>cy.reservarCancha(id)</code>	Buscar la cancha por su <code>data-id</code> y hacer clic en su botón «Reservar»

PISTA

La regla para saber si un comando propio vale la pena.

Si esconde **una intención del usuario** —«entrar», «reservar la cancha X»—, hazlo.

Si solo esconde **un selector** —`cy.botonEntrar()`—, no: estás inventando un diccionario que nadie va a recordar y que tapa lo que el test hace de verdad.

2

Después: la regla de las dos reservas

Ahora los tres escenarios. Son pocos y pesados: acá se juega la plata del complejo.

#	SITUACIÓN	QUÉ TIENES QUE COMPROBAR
1	Reservo la cancha <code>C-01</code>	Desaparece de la lista, quedan 3 canchas y el contador dice « 1 de 2 »
2	Reservo <code>C-01</code> y después <code>C-02</code>	Quedan 2 canchas, el contador dice « 2 de 2 », aparece el aviso del límite y todos los botones «Reservar» quedan deshabilitados
3	Reservo <code>C-03</code> y recargo la pantalla de canchas	<code>C-03</code> sigue sin aparecer : la reserva quedó guardada de verdad, no solo en pantalla

CUIDADO

Uno de estos tres se te va a poner rojo. Es a propósito.

Cuando pase: **no toques el test**. El test está diciendo la verdad — la aplicación no cumple la regla que el complejo prometió. Arregla `VistaCanchas.vue`, vuelve a correr, y cuenta en el README qué estaba mal, qué provocaba y cuál de tus escenarios lo delató.

Un test que se arregla bajándole la exigencia no es un test: es un adorno.

PISTA

Pista sobre el escenario 3, que es el que más gente escribe mal.

Si recargas la pantalla apenas haces clic, la petición de reserva puede ir todavía en camino y el servidor no alcanza a enterarse: recargas, la cancha vuelve a aparecer, y tu test falla acusando a la aplicación de algo que no hizo.

Antes de recargar, **asegúrate de que el servidor recibió la orden**. Ya sabes cómo: es lo mismo que hiciste en el Requisito 4 con las peticiones de canchas, pero ahora sobre `POST /api/canchas/*/reservar`. El `*` significa «cualquier cosa en ese tramo de la dirección».

3

Criterios de aceptación

CRITERIOS

Se revisa esto, uno por uno

- ☐ Existen los tres comandos y están en `cypress/support/commands.js`, no copiados en cada spec.
- ☐ `cypress/support/e2e.js` importa `./commands.js`.
- ☐ **Reescribiste los requisitos 2, 3 y 4** para que usen `cy.entrar()` y `cy.reiniciarDatos()`. No queda ni un acceso escrito a mano fuera del `02-acceso.cy.js`.
- ☐ El escenario 2 comprueba que están deshabilitados **todos** los botones que quedan, no solo el primero.
- ☐ El escenario 3 espera la confirmación del servidor antes de recargar.
- ☐ El defecto está arreglado **en** `VistaCanchas.vue`, no tapado en el test.
- ☐ El README explica el defecto en dos líneas: qué estaba mal, qué provocaba y qué escenario lo pilló.
- ☐ La suite completa pasa: 22 escenarios más los 2 del ejemplo.

CLAVE

Por qué el escenario 2 vale por toda la actividad.

El acceso, el filtro y el estado vacío son comodidad. Esto es **plata**: si la regla de las dos canchas no se cumple, tres socios se llevan el recinto entero un sábado y el resto se queda afuera. Ese tipo de fallo no lo pilla nadie mirando la pantalla, porque *parece* que todo anda bien.

1 Qué entregas

ARCHIVO	QUÉ CONTIENE	ESCENARIOS
<code>cypress/e2e/00-ejemplo.cy.js</code>	El que te dimos	2
<code>cypress/e2e/01-primeros-pasos.cy.js</code>	Abrir y mirar	4
<code>cypress/e2e/02-acceso.cy.js</code>	El trámite de entrar y salir	5
<code>cypress/e2e/03-lista.cy.js</code>	La lista, el filtro y el estado vacío	5
<code>cypress/e2e/04-servidor.cy.js</code>	Error, vacío, lento y recuperación	5
<code>cypress/e2e/05-reservas.cy.js</code>	La regla de las dos reservas	3
<code>cypress/support/commands.js</code>	Tus tres comandos propios	—
<code>cypress/fixtures/canchas-tarde.json</code>	Los datos falsos del R4	—
<code>src/vistas/VistaCanchas.vue</code>	Con el defecto arreglado	—
<code>README.md</code>	Los tres hallazgos	—

CLAVE

Lo que tiene que decir el README. Tres puntos, dos líneas cada uno.

1. El **defecto del R5**: qué estaba mal, qué provocaba en la vida real, cómo lo arreglaste y cuál de tus escenarios lo delató.
2. Qué pasó cuando probaste el **estado vacío** del R3, y por qué una pantalla en blanco no es lo mismo que una pantalla que avisa.
3. Un escenario que **quisiste escribir y no pudiste**, y por qué. Si no hubo ninguno, dilo — pero piénsalo antes.

#	COMPROBACIÓN	CÓMO SE HACE
1	La suite completa pasa en verde	<code>npm run cy:run</code> : 24 escenarios, 0 fallos
2	El defecto está arreglado en la aplicación, no tapado en el test	Leer la comparación del límite en <code>VistaCanchas.vue</code>
3	Cero esperas por tiempo	Buscar <code>cy.wait()</code> seguido de un número: no puede aparecer ninguna
4	Cero selectores frágiles	Buscar <code>nth-child</code> , <code>.btn</code> y clases CSS en <code>cypress/e2e/</code>
5	Los escenarios son independientes	Correr un archivo suelto: <code>npx cypress run --spec cypress/e2e/03-lista.cy.js</code>
6	Cada archivo deja el sistema en estado conocido	Que todos tengan <code>cy.reiniciarDatos()</code> en su <code>beforeEach</code>
7	Las intercepciones están antes de la acción	Que ningún <code>cy.intercept</code> aparezca después de su <code>cy.visit</code>
8	Los comandos propios se usan de verdad	Buscar <code>data-cy="correo"</code> fuera de <code>02-acceso.cy.js</code> : no puede aparecer
9	Los nombres describen comportamiento	Leer los <code>it</code> sin mirar el código: ¿se entiende qué hace el socio?
10	La suite corre dos veces seguidas con el mismo resultado	Ejecutarla dos veces sin reiniciar el servidor

SÍNTOMA	CAUSA	ARREGLO
« <i>cy.visit() failed trying to load</i> »	La aplicación no está levantada	<code>npm run dev</code> en la otra terminal, y no la cierras
« <i>Port 5173 is in use</i> »	Tienes otro Vite dando vueltas	Ciérralo, o cambia <code>baseUr1</code> y <code>server.port</code> a la vez
No aparece ningún archivo de test en Cypress	Tu archivo no termina en <code>.cy.js</code> o está fuera de <code>cypress/e2e/</code>	Revisa el nombre y la carpeta
« <i>no request ever occurred</i> »	Interceptaste después de que salió la petición	<code>cy.intercept</code> siempre antes del <code>cy.visit</code>
La pantalla revienta al interceptar	Le entregaste un arreglo donde la aplicación espera un objeto	<code>{ body: { canchas: [], reservasDelDia: 0 } }</code>
« <i>expected to find option with value tenis</i> »	Le pasaste a <code>.select()</code> el texto visible en vez del valor	El valor va en minúscula: <code>padel</code> , <code>tenis</code>
El escenario del «Cargando...» nunca lo alcanza a ver	El servidor de clase contesta en 2 ms	<code>delay: 1500</code> en la intercepción
Un escenario pasa solo pero falla en grupo	El anterior reservó una cancha y no reiniciaste	<code>cy.reiniciarDatos()</code> en el <code>beforeEach</code>
« <i>element is detached from the DOM</i> »	Encadenaste dos búsquedas y la lista se volvió a dibujar en el medio	Una sola búsqueda que apunte directo a lo que te interesa
<code>cy.entrar is not a function</code>	<code>e2e.js</code> no importa <code>commands.js</code>	Agregar <code>import './commands.js'</code>
La comprobación ve la pantalla anterior	Guardaste el resultado en una variable en vez de encadenar	La comprobación va pegada con un punto al <code>cy.get</code>

SÍNTOMA	CAUSA	ARREGLO
Un test nunca falla, ni cuando debería	Escribiste <code>.should</code> sin paréntesis, o afirmaste sobre nada	Rómpelo a propósito y comprueba que se pone rojo

4 La prueba de que tus tests sirven

CUIDADO

Rómpelos a propósito. Esto es parte de la entrega, no un extra.

Cuando tengas los 24 en verde, entra a `VistaCanchas.vue` y haz **uno por uno** estos tres destrozos, corriendo la suite después de cada uno y devolviendo el cambio antes del siguiente:

1. Borra el `data-cy="vacío"` del párrafo del estado vacío.
2. Cambia el mensaje de error por `'Error 500'`.
3. Saca el `:disabled` del botón «Reservar».

Si alguno de los tres no pone nada rojo, ese comportamiento no estaba probado. Devuelve el cambio y escribe el escenario que faltaba. Un test que sigue verde con la aplicación rota es peor que no tener test, porque le enseña al equipo a confiar en algo que no mira.

5 Si terminas antes

SI TERMINAS ANTES

Extras, en orden de dificultad

1. **Lo que se envía, no solo lo que llega.** Escribe un escenario que compruebe que la pantalla de acceso manda al servidor exactamente `{ correo, clave }`, con esos dos nombres y sin espacios de más. Pista: `cy.wait('@alias')` no solo espera, también te entrega la petición completa.
2. **El botón que no se bloquea.** Con `delay` en la reserva, comprueba qué pasa si el socio hace clic dos veces seguidas en «Reservar» antes de que llegue la

respuesta. ¿Se cuenta una reserva o dos? Documenta lo que encuentres — no tienes que arreglarlo.

3. **El teléfono.** Agrega `cy.viewport('iphone-x')` a un escenario del R3 y comprueba que la lista sigue usable en pantalla chica. Si algo se rompe, anótalo.
4. **Una sola orden.** Instala `start-server-and-test` y deja un script `npm run e2e` que levante Vite, corra la suite y apague todo. Es la pieza que usa un equipo de verdad para correr esto de forma automática.
5. **La pregunta incómoda.** El límite de dos reservas lo controla la *pantalla*; el servidor acepta todas las que le manden. Escribe un `cy.request` que pida tres reservas **sin pasar por la interfaz** y mira qué contesta. Explica en el README por qué eso es un problema y de quién es la responsabilidad de arreglarlo.

EL ENCARGO

Lo que estás demostrando con esta entrega.

Que sabes traducir **una regla del negocio** —«máximo dos canchas por socio»— a un escenario que la vigila sola, todos los días, sin que nadie se acuerde de revisarla. Y que sabes probar los caminos que nadie mira: el error, el vacío y la espera.

Veintidós escenarios que corren en segundos y que se ponen rojos *cuando de verdad hay algo roto*. Eso es lo que se entrega.